DEEPFAKE DETECTION

High-Level Design

Design Choices, Rationale, and MLOps Pipeline

Major Prabhat Pandey | April 2026 | MLOps Project

1. System Overview

The Deepfake Detection System is an end-to-end ML platform that classifies MP4 videos as real or fake using a deep learning model served through a production-grade MLOps pipeline. The system is designed for local deployment using Docker Compose.

Key design goals:

- End-to-end deepfake classification with ≥ 90% accuracy and ≥ 0.90 F1-score
- Full MLOps lifecycle: data versioning, experiment tracking, model registry, automated retraining
- Sub-200ms inference latency at p95
- Reproducibility: every experiment tied to a Git commit hash and MLflow run ID
- Monitoring, alerting, and drift detection on all components

2. MLOps Pipeline

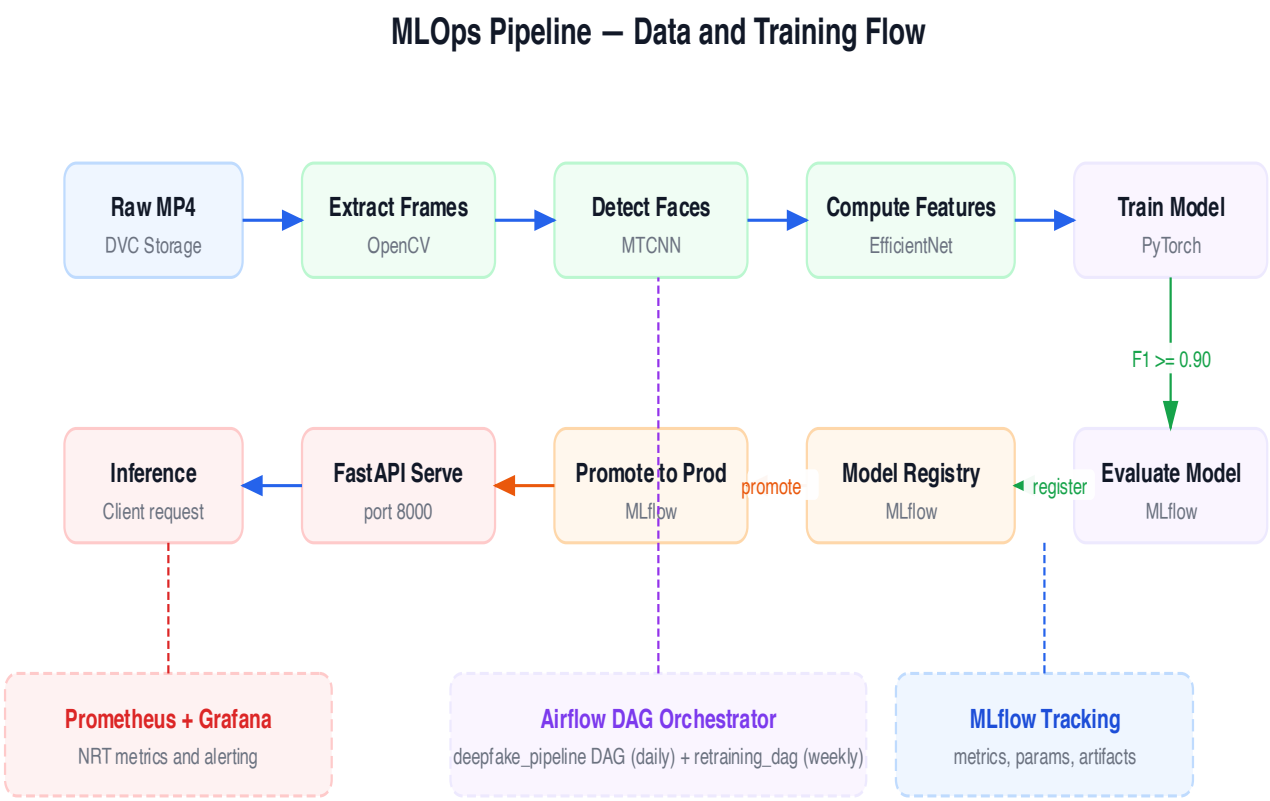

Figure 2: MLOps Data and Training Pipeline

The pipeline flows in two paths: a data processing path (top row) that feeds raw MP4 videos through frame extraction, face detection, and feature computation, and a model lifecycle path (bottom row) that handles training, evaluation, registry, and promotion.

2.1 Data Engineering Pipeline (Airflow — Daily)

The deepfake_pipeline DAG runs daily and executes these stages in sequence:

Stage	Tool	Input	Output
ingest_videos	Airflow PythonOperator	data/landing/*.mp4	data/raw/*.mp4
extract_frames	OpenCV	data/raw/	data/frames/ (30 JPGs/video)
detect_faces	MTCNN	data/frames/	data/faces/ (cropped 224×224)
compute_features	EfficientNet-B0	data/faces/	data/features/*.pt
validate_schema	Custom validator	data/features/	validation report
record_baseline_stats	NumPy	data/features/	ml/feature_baseline.json
version_with_dvc	DVC	all data/	dvc.lock committed to Git

2.2 DVC CI Pipeline

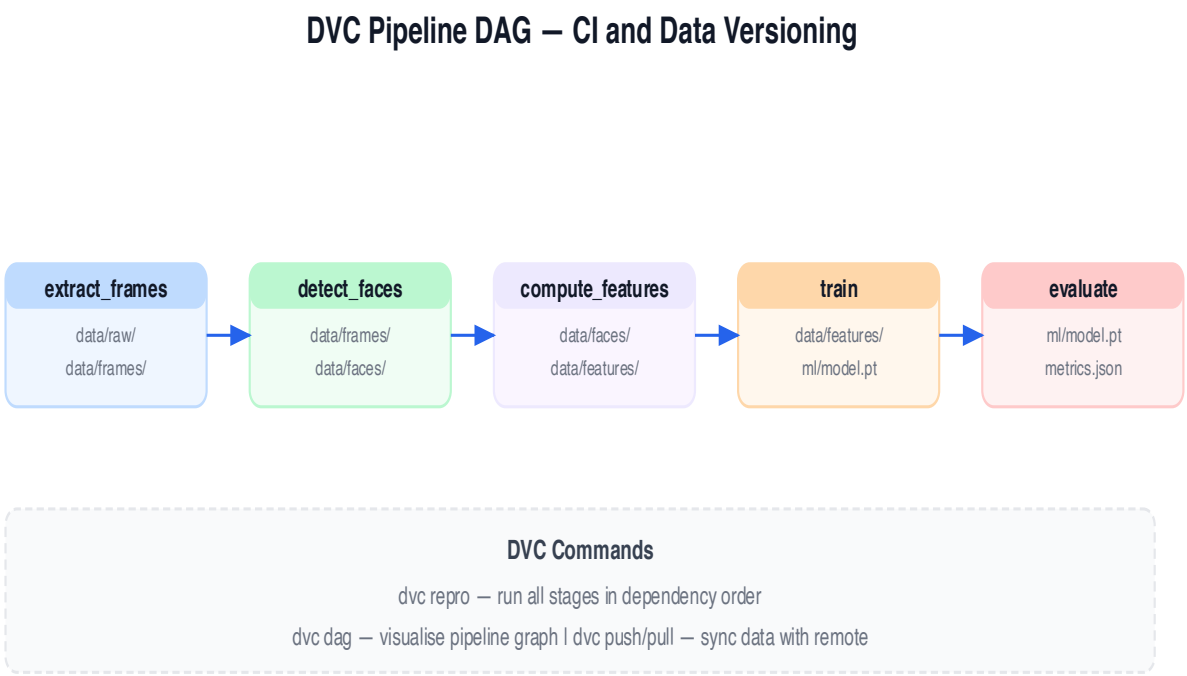

Figure 3: DVC Pipeline DAG — Five Reproducible Stages

DVC defines a five-stage pipeline in dvc.yaml with explicit dependency declarations. Running dvc repro re-executes only stages whose inputs have changed, making the pipeline fully reproducible and auditable. The dvc.lock file is committed to Git, ensuring any team member can recreate the exact pipeline state.

2.3 Automated Retraining (Airflow — Weekly)

The retraining_dag uses a BranchPythonOperator to decide whether retraining is needed based on feature drift. If drift score exceeds 3.0, the full retraining pipeline executes:

- fetch_new_data: pulls newly labelled data from data/landing/
- run_mlproject: executes ml/MLproject with params from ml/params.yaml
- evaluate_model: computes F1, accuracy, PR-AUC, per-class precision/recall
- register_if_better: registers model to MLflow registry if F1 ≥ 0.90
- promote_to_production: transitions Staging → Production, archives current Production

3. Model Architecture

The model is a CNN + LSTM architecture using EfficientNet-B0 as a temporal feature extractor:

```
Input: (batch, 30, 3, 224, 224)  — 30 face-cropped frames
EfficientNet-B0 (pretrained ImageNet)  — spatial features per frame
2-layer LSTM (hidden=256)  — temporal dependencies across frames
Linear(256→128) → ReLU → Dropout(0.3) → Linear(128→1) → Sigmoid
Output: (batch, 1)  — probability (>= 0.5 = fake)
```

Decision	Choice	Rationale
Feature extractor	EfficientNet-B0	Best accuracy/efficiency trade-off; pretrained ImageNet weights reduce training time
Temporal model	2-layer LSTM	Captures inter-frame temporal dependencies; simpler than transformers for 30-frame sequences
Face detection	MTCNN	High accuracy, handles pose variation; lightweight at inference
Experiment tracking	MLflow	Rich model registry with stage transitions; pyfunc serving eliminates extra container
Data pipeline	Airflow + DVC	Airflow for orchestration and scheduling; DVC for reproducible data versioning
Model serving	mlflow.pyfunc (in-process)	Backend loads model directly — no extra container, lower latency, fewer failure points
Monitoring	Prometheus + Grafana	Industry standard; no external dependencies; full metrics lifecycle
Frontend	React + Vite + TypeScript	Fast build; type safety; component-based architecture

4. Experiment Tracking

MLflow tracking is fully manual — no autolog — to give precise control over what is recorded:

What is Tracked	How	Purpose
All hyperparameters	mlflow.log_params(params)	Single source of truth from ml/params.yaml
Per-epoch metrics	mlflow.log_metrics({...}, step=epoch)	train_loss, val_loss, train_f1, val_f1, val_accuracy, learning_rate
Git commit SHA	mlflow.set_tag("git_commit", ...)	Links every run to the exact code state
Device used	mlflow.set_tag("device", ...)	CPU vs CUDA affects reproducibility
Best model artifact	mlflow.pytorch.log_model()	Registered to Model Registry as 'deepfake'
Confusion matrix	mlflow.log_artifact()	Visual evaluation artifact
ROC curve	mlflow.log_artifact()	AUC-ROC visual artifact
Extended eval metrics	mlflow.log_metrics()	PR-AUC, per-class precision/recall, optimal threshold

5. Monitoring Architecture

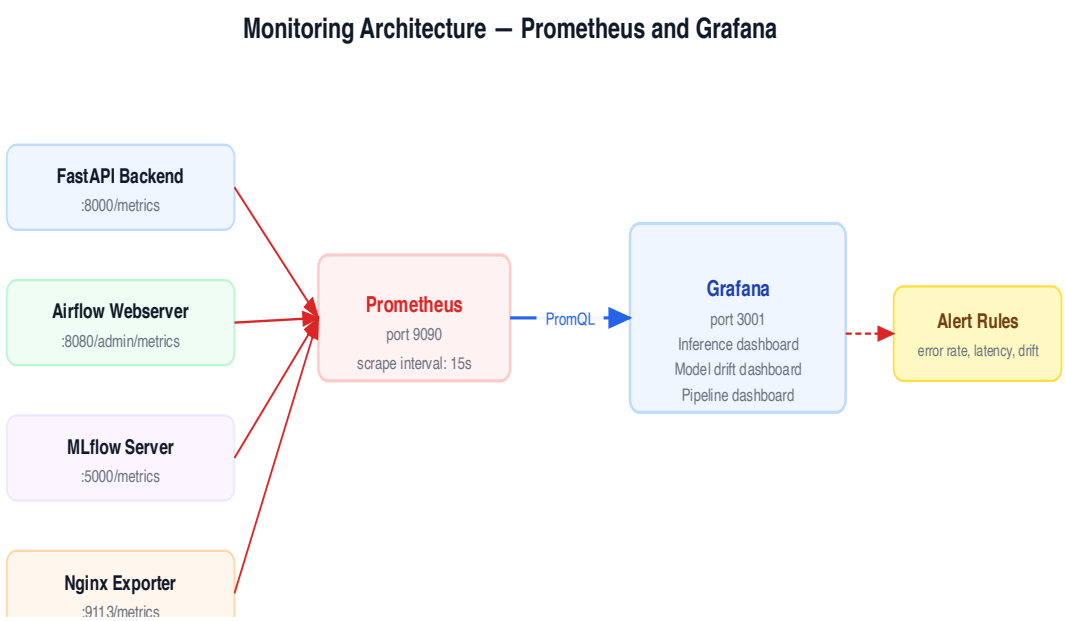

Figure 4: Prometheus and Grafana Monitoring Architecture

All components expose Prometheus metrics endpoints. Prometheus scrapes every 15 seconds. Grafana visualises three dashboards in near-real-time:

- Inference Dashboard: request rate, latency histogram, error rate, drift score gauge
- Model Drift Dashboard: z-score time series with threshold markers at 3.0
- Pipeline Dashboard: Airflow task durations, DVC pipeline throughput, data volume

Alert rules fire when:

Condition	Threshold	Response
Error rate	> 5% for 3 minutes	Grafana alert + investigate logs
p95 Latency	> 200ms	Grafana alert + profile backend
Drift score	> 3.0	Triggers retraining_dag BranchPythonOperator
Pipeline validation failure	Any failure	Grafana alert + check Airflow logs

6. Software Design Paradigm

The codebase uses a mixed OO + functional paradigm, chosen per layer:

Layer	Paradigm	Rationale
ML model (ml/model.py)	Object-Oriented — DeepfakeDetector(nn.Module)	PyTorch Module system requires OO; encapsulates CNN + LSTM as a single composable unit
Backend API (routers/)	Functional — FastAPI route functions	Stateless request handlers are naturally functional; easier to test in isolation
Model loader	Module-level singleton	Holds shared mutable state (loaded model) without a class; standard Python pattern for process-wide singletons
Airflow DAGs	Functional — plain Python callables	Airflow PythonOperator expects callables; pure functions are easier to test
Preprocessing	Functional with lazy singleton	Stateless transforms; MTCNN cached via module-level variable to avoid reloading per request

7. Security

- All secrets stored in .env (never committed to Git; .env.example provided)
- HTTPS via Nginx TLS termination in production
- Prometheus and Grafana behind basic auth
- No cloud usage — all data stays local (GDPR-safe for video PII)
- Encryption at rest: Docker volumes mounted on OS-level encrypted filesystem (BitLocker/LUKS)
- Admin endpoint (POST /admin/reload-model) not exposed through Nginx to public users